RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name : KARTHIKEYAN M

Year / Branch / Section : 2nd Year IT- A

University Register No. : 2116251001105

College Roll No. : 251001105

Semester : 3

Academic Year : 2025-2026

INDEX PAGE

Exp. No	Name of the Experiment	Date of Completion
1.	Finding Complexity using Counter Method	06/08/2026
2.	Finding Complexity using Counter Method	06/08/2026
3.	Finding Complexity using Counter Method	06/08/2026
4.	Finding Complexity using Counter Method	13/08/2026
5.	Finding Complexity using Counter Method	20/08/2026
6.	1-G-Coin Problem	27/08/2026
7.	2-G-Cookies Problem	27/08/2026
8.	3-G-Burger Problem	14/09/2026
9.	4-G-Array Sum max problem	16/09/2026
10.	5-G-Product of Array elements-Minimum	16/09/2026
11.	Number of Zeros in a Given Array	20/08/2026
12.	Majority Element	20/08/2026
13.	Finding Floor Value	20/08/2026
14.	Two Elements sum to x	20/08/2026
15.	Implementation of Quick Sort	25/08/2026
16.	DP-Playing with Numbers	02/10/2026
17.	DP-Playing with chessboard	02/10/2026
18.	DP-Longest Common Subsequence	02/10/2026
19.	DP-Longest non-decreasing Subsequence	02/10/2026
20.	Finding Duplicates-$O(n^2)$ Time Complexity, O(1) Space Complexity	17/09/2026
21.	Finding Duplicates-O(n) Time Complexity, O(1) Space Complexity	17/09/2026
22.	Print Intersection of 2 sorted arrays-O(m*n) Time Complexity, O(1) Space Complexity	24/09/2026
23.	Print Intersection of 2 sorted arrays-O(m+n) Time Complexity, O(1) Space Complexity	24/09/2026
24.	Pair with Difference-$O(n^2)$ Time Complexity, O(1) Space Complexity	24/09/2026
25.	Pair with Difference-O(n) Time Complexity, O(1) Space Complexity	24/09/2026

1.a) Finding Complexity using Counter Method

AIM:

To implement the given algorithm as a program and determine its time complexity using the counter method.

ALGORITHM:

1. Start the program and initialize i = 1, s = 1, and count = 0.
2. Read the value of n from the user.
3. Check whether s <= n; if true, increment count and i.
4. Add i to s and update the counter for each operation; repeat until the condition becomes false.
5. Print the total counter value and stop.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n;
   int i = 1, s = 1, count = 0;
   scanf("%d", &n);
   count++;
   count++;
   while (s <= n)
   {
      count++;
      i++;
      count++;
      s = s + i;
      count++;
   }
   count++;
   printf("%d", count);
   return 0;
}
```

OUTPUT:

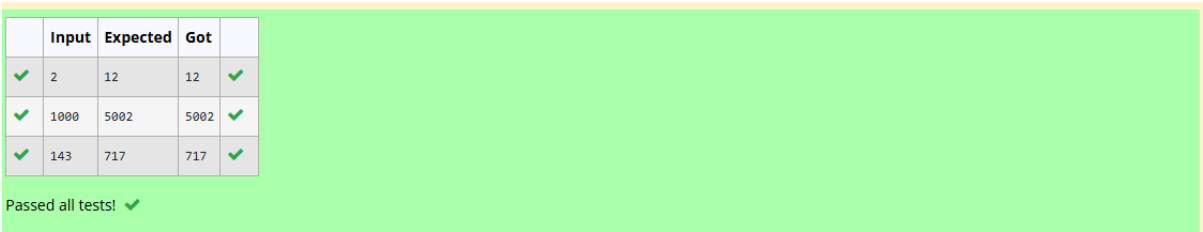
	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

RESULT:

The given algorithm was implemented successfully using C programming, and its time complexity was analyzed using the counter method.

OUTPUT:



	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

1.b) Finding Complexity using Counter method

AIM:

To implement the given algorithm in C and determine its time complexity using the counter method.

ALGORITHM:

1. Start the program and read the value of n.
2. If n == 1, print *.
3. Otherwise, execute the outer loop from 1 to n.
4. Execute the inner loop, print **, and terminate the inner loop using break.
5. Count the operations, display the result, and stop.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n, count = 0;
   scanf("%d", &n);
   if (n == 1)
   {
      count++;
   }
   else
   {
      count++;
      for (int i = 1; i <= n; i++)
      {
         count++;
         for (int j = 1; j <= n; j++)
         {
            count++;
```

```
count++;
            count++;
            break;
        }
        count++;
    }
    count++;
  }
  printf("%d", count);
  return 0;
}
```

RESULT:

The given algorithm was implemented successfully in C, and its time complexity was determined using the counter method as \(O(n)\).

OUTPUT:

	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

1.c) Finding Complexity using Counter Method

AIM:

To write a C program to find and print all factors of a given positive integer and determine its time complexity using the counter method.

ALGORITHM:

1. Start the program and read the positive integer num.
2. Initialize the counter variable count = 0.
3. Execute a for loop from i = 1 to num.
4. Increment count and check whether num % i == 0. If true, print i.
5. Print the counter value and stop the program.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int num, count = 0;
   scanf("%d", &num);
   for (int i = 1; i <= num; ++i)
   {
      count++;
      count++;
      if (num % i == 0)
      {
         count++;
      }
   }
   count++;
   printf("%d", count);
   return 0;
}
```

RESULT:

The C program was implemented successfully to find the factors of a positive integer. The time complexity was calculated using the counter method as \(O(n)\).

OUTPUT:

	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

1.e) Finding Complexity using Counter Method

AIM:

To implement the given algorithm in C and determine its time complexity using the counter method.

ALGORITHM:

1. Start the program and read the value of n.
2. Initialize the counter variable c = 0.
3. Execute the outer loop from n/2 to n-1.
4. Execute the inner loops by doubling j and k in each iteration, and increment c.
5. Print the counter value and stop the program.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n, c = 0, count = 0;
   scanf("%d", &n);
   count++;
   for (int i = n / 2; i < n; i++)
   {
      count++;
      for (int j = 1; j < n; j = 2 * j)
      {
         count++;
         for (int k = 1; k < n; k = k * 2)
         {
            count++;
            c++;
            count++;
         }
         count++;
      }
```

```
count++;
    }
    count++;
    printf("%d", count);
    return 0;
}
```

RESULT:

The given algorithm was implemented successfully in C, and its time complexity was determined using the counter method as $O(n\log^2 n)$.

OUTPUT:

	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

1.e) Finding Complexity using counter method

AIM:

To write a C program to reverse a given integer and determine its time complexity using the counter method.

ALGORITHM:

1. Start the program and read the integer n.
2. Initialize rev = 0 and declare remainder.
3. Repeat while n != 0: calculate remainder = n % 10.
4. Update rev = rev * 10 + remainder and divide n by 10.
5. Print rev and the counter value, then stop.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n, rev = 0, remainder, count = 0;
   scanf("%d", &n);
   count++;
   while (n != 0)
   {
      count++;
      remainder = n % 10;
      count++;
      rev = rev * 10 + remainder;
      count++;
      n /= 10;
      count++;
   }
   count++;
   count++;
```

```
printf("%d", count);
    return 0;
}
```

RESULT:

The C program was implemented successfully to reverse an integer. Using the counter method, its time complexity was determined as $O(\log n)$, where n is the input number.

OUTPUT:

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

GREEDY ALGORITHMS

2.a) 1-G-Coin Problem

AIM:

To write a C program to find the minimum number of Indian currency notes and coins required to make a given amount using the greedy algorithm.

ALGORITHM:

1. Start the program and read the amount V.
2. Initialize count = 0 and store the denominations in descending order.
3. Select the largest denomination that is less than or equal to the remaining amount.
4. Add V / denomination to count and update V using V % denomination.
5. Repeat for all denominations, print count, and stop.

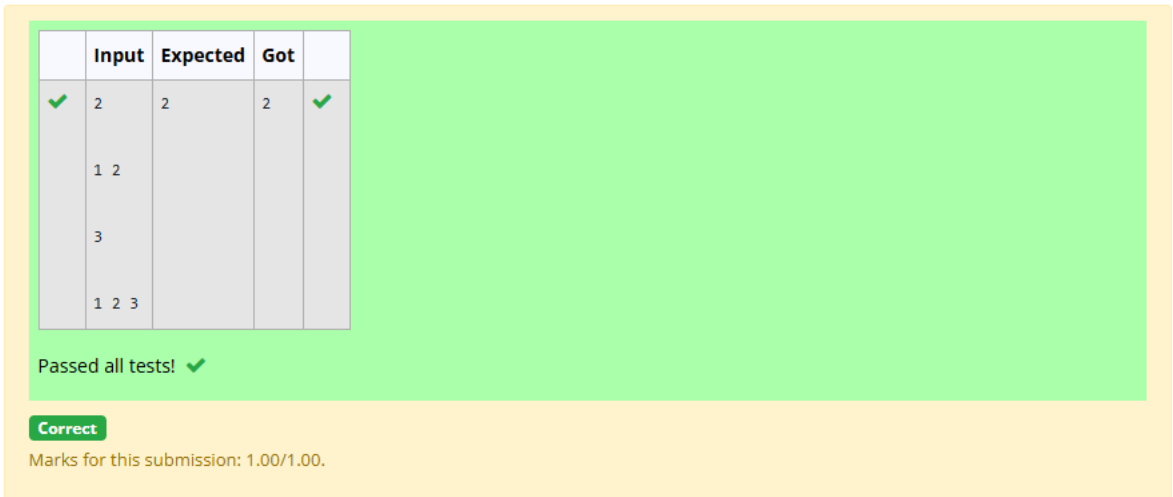
PROGRAM:

```
#include <stdio.h>
int main()
{
   int V, count = 0;
   int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
   int i;
   scanf("%d", &V);
   for (i = 0; i < 9; i++)
   {
      count += V / coins[i];
      V = V % coins[i];
   }
   printf("%d", count);
   return 0;
}
```

RESULT:

The program was implemented successfully to find the minimum number of Indian currency notes and coins required to make the given amount.

OUTPUT:



	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2.b) 2-G-Cookies Problem

AIM:

To write a C program to assign cookies to children based on their greed factors and maximize the number of content children using the greedy algorithm.

ALGORITHM:

1. Start the program and read the number of children and cookies.
2. Read the greed factors of the children and the sizes of the cookies.
3. Sort both arrays in ascending order.
4. Compare each child's greed factor with the cookie size and assign a cookie if its size is sufficient.
5. Count and print the maximum number of content children, then stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int compare(const void *a, const void *b)
{
   return (*(int *)a - *(int *)b);
}
int main()
{
   int n, m, i, j = 0, count = 0;
   scanf("%d", &n);
   int g[n];
   for (i = 0; i < n; i++)
      scanf("%d", &g[i]);
   scanf("%d", &m);
   int s[m];
   for (i = 0; i < m; i++)
      scanf("%d", &s[i]);
   qsort(g, n, sizeof(int), compare);
```

```
    qsort(s, m, sizeof(int), compare);
    i = 0;
    while (i < n && j < m)
    {
        if (s[j] >= g[i])
        {
            count++;
            i++;
        }
        j++;
    }
    printf("%d", count);
    return 0;
}
```

RESULT:

Thus the program was implemented successfully to maximize the number of content children using the greedy algorithm.

OUTPUT:

	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2.c) 3-G-Burger Problem

AIM:

To write a C program to determine the minimum distance a person needs to run after eating burgers using the greedy approach and an efficient sorting algorithm.

ALGORITHM:

1. Start the program and read the number of burgers n.
2. Read the calorie values of all burgers into an array.
3. Sort the calorie values in descending order using an efficient sorting algorithm.
4. Calculate the total distance by adding each calorie value multiplied by \(3^i\), where i starts from 0.
5. Print the minimum running distance and stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b)
{
   long long x = *(const long long *)a;
   long long y = *(const long long *)b;
   if (x < y) return 1;
   if (x > y) return -1;
   return 0;
}
int main()
{
   int n;
   scanf("%d", &n);
   long long *c = (long long *)malloc((n > 0 ? n : 1) * sizeof(long long));
   for (int i = 0; i < n; i++)
      scanf("%lld", &c[i]);
   qsort(c, n, sizeof(long long), cmp);
```

```
  long long total = 0, mult = 1;
   for (int i = 0; i < n; i++)
   {
      total += mult * c[i];
      mult *= n;
   }
   printf("%lld", total);
   free(c);
   return 0;
}
```

RESULT:

Thus the program was designed to determine the minimum running distance using the greedy approach by eating burgers in descending order of calorie values.

OUTPUT:

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2.d) 4-G-Array Sum max problem

AIM:

To write a program using the greedy technique to maximize the sum of arr[i] * i by arranging the array elements optimally in time.

ALGORITHM:

1. Start the program and read the number of elements n.
2. Read the n array elements.
3. Sort the array in ascending order using an efficient sorting algorithm.
4. Calculate the sum of arr[i] * i for all indices from 0 to n-1.
5. Print the maximum array sum and stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b)
{
   long long x = *(const long long *)a;
   long long y = *(const long long *)b;
   if (x < y) return -1;
   if (x > y) return 1;
   return 0;
}
int main()
{
   int n;
   scanf("%d", &n);
   long long *arr = (long long *)malloc((n > 0 ? n : 1) * sizeof(long long));
   for (int i = 0; i < n; i++)
      scanf("%lld", &arr[i]);
   qsort(arr, n, sizeof(long long), cmp);
```

```
  long long sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i] * i;
    printf("%lld", sum);
    free(arr);
    return 0;
}
```

RESULT:

Thus the program was designed using the greedy technique to maximize the array sum. The time complexity is \(O(n\log n)\), dominated by sorting.

OUTPUT:

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 60 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

2.e) 5-G-Product of Array elements - Minimum

AIM:

To write a program using the greedy technique to rearrange two arrays of equal size such that the sum of the products of corresponding elements is minimized.

ALGORITHM:

1. Start the program and read the size N of both arrays.
2. Read the elements of array_One[] and array_Two[].
3. Sort the first array in ascending order and the second array in descending order.
4. Multiply the corresponding elements of both arrays and calculate their total sum.
5. Print the minimum sum and stop.

PROGRAM:

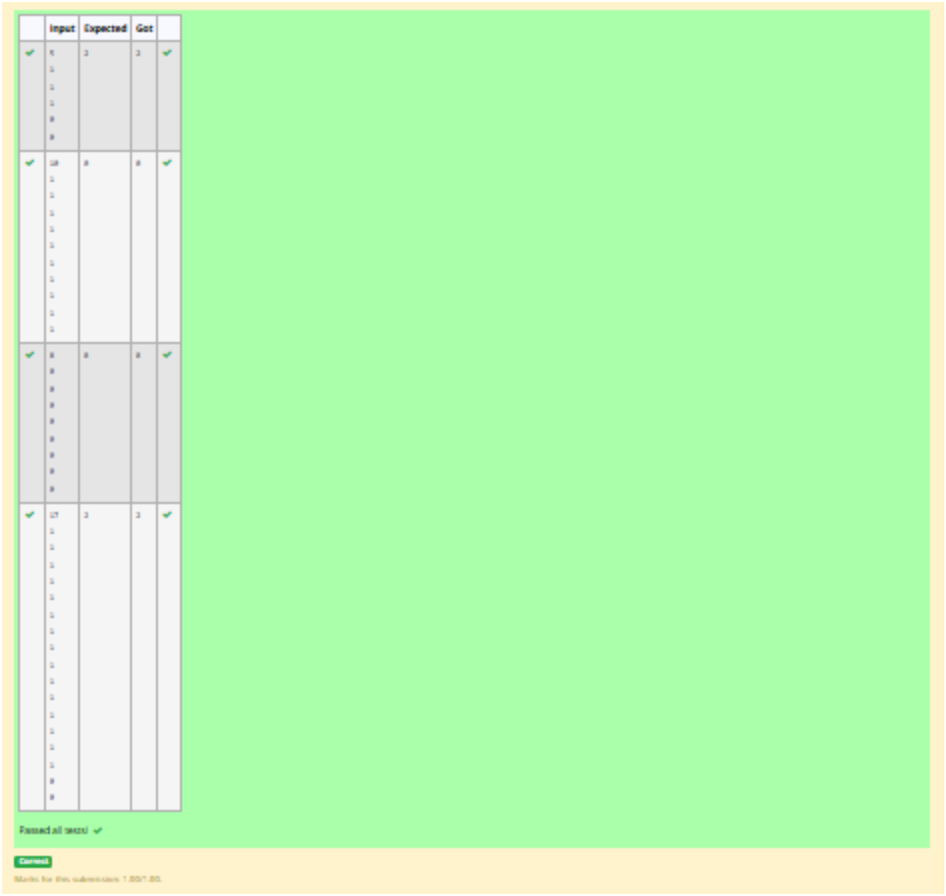
```
#include <stdio.h>
#include <stdlib.h>
int asc(const void *a, const void *b)
{
   long long x = *(const long long *)a;
   long long y = *(const long long *)b;
   if (x < y) return -1;
   if (x > y) return 1;
   return 0;
}
int desc(const void *a, const void *b)
{
   return asc(b, a);
}
int main()
{
   int n;
   scanf("%d", &n);
   long long *A = (long long *)malloc((n > 0 ? n : 1) * sizeof(long long));
```

```
 long long *B = (long long *)malloc((n > 0 ? n : 1) * sizeof(long long));
    for (int i = 0; i < n; i++)
        scanf("%lld", &A[i]);
    for (int i = 0; i < n; i++)
        scanf("%lld", &B[i]);
    qsort(A, n, sizeof(long long), asc);
    qsort(B, n, sizeof(long long), desc);
    long long sum = 0;
    for (int i = 0; i < n; i++)
        sum += A[i] * B[i];
    printf("%lld", sum);
    free(A);
    free(B);
    return 0;
}
```

RESULT:

Thus the program was designed using the greedy technique to minimize the sum of products of corresponding array elements by sorting the arrays in opposite orders.

OUTPUT:



DIVIDE AND CONQUER

3.a) Number of Zeros in a Given Array

AIM:

To write a C program using the Divide and Conquer technique to count the number of zeros in an array containing all 1s followed by all 0s.

ALGORITHM:

1. Start the program and read the size n and array elements.
2. Find the middle index of the array.
3. If the middle element is 1, search the right half; otherwise, search the left half.
4. Repeat the process using Divide and Conquer to locate the first zero.
5. Count and print the number of zeros, then stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int findFirstZero(int arr[], int low, int high)
{
   if (low > high)
   {
      return -1;
   }
   if (arr[low] == 0)
   {
      return low;
   }
   if (arr[high] == 1)
   {
      return -1;
   }

  int mid = low + (high - low) / 2;
```

```
if (arr[mid] == 0)
   {
      return findFirstZero(arr, low, mid);
   }
   else
   {
      return findFirstZero(arr, mid + 1, high);
   }}
int main()
{
   int m;
   scanf("%d", &m);
   int *arr = (int *) malloc(m * sizeof(int));
   int i;
   for (i = 0; i < m; i++)
   {
      scanf("%d", &arr[i]);}
   int firstZeroIndex = findFirstZero(arr, 0, m - 1);
   int countOfZeroes;
   if (firstZeroIndex == -1)
   {countOfZeroes = 0;
   }
   else
   {    countOfZeroes = m - firstZeroIndex;   }
   printf("%d\n", countOfZeroes);
   free(arr);
  return 0;}
```

Result

Thus the program was designed using the Divide and Conquer technique to count the zeros in the given array efficiently.

OUTPUT:

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3.b) -Majority Element

AIM:

To write a C program to find the majority element in an array, which appears more than \(\lfloor n/2 \rfloor\) times, using an efficient algorithm.

ALGORITHM:

1. Start the program and read the size n and array elements.
2. Initialize a candidate element and a counter.
3. Traverse the array using the Boyer–Moore Voting Algorithm.
4. Update the candidate and counter based on the current element.
5. Print the majority element and stop.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n;
   scanf("%d", &n);
   int candidate = 0, count = 0;
   for (int i = 0; i < n; i++)
   {
      int x;
      scanf("%d", &x);
      if (count == 0)
         candidate = x;
      if (x == candidate)
         count++;
      else
         count--;
   }
   printf("%d", candidate);
   return 0;}
```

RESULT:

Thus the program was designed successfully to find the majority element using the Boyer–Moore Voting Algorithm with \(O(n)\) time complexity and \(O(1)\) extra space.

OUTPUT:

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3.c) -Finding Floor Value

AIM:

To write a program using the Divide and Conquer technique to find the floor of a given value x in a sorted array.

ALGORITHM:

1. Start the program and read the sorted array and the value x.
2. Find the middle element of the array.
3. If the middle element is equal to x, return it.
4. If it is smaller than x, search the right half; otherwise, search the left half.
5. Print the largest element less than or equal to x, or report that no floor exists.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int floorSearch(int arr[], int low, int high, int x)
{
   if (low > high)
      return -1;
   int mid = low + (high - low) / 2;
   if (arr[mid] == x)
     return arr[mid];

   if (arr[mid] < x)
   {
      int r = floorSearch(arr, mid + 1, high, x);
      return (r != -1) ? r : arr[mid];
   }
   return floorSearch(arr, low, mid - 1, x);
}
int main()
{
```

```
    int n, x;
    scanf("%d", &n);
    int *arr = (int *)malloc((n > 0 ? n : 1) * sizeof(int));
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    scanf("%d", &x);
    printf("%d", floorSearch(arr, 0, n - 1, x));
    free(arr);
    return 0;
}
```

RESULT:

Thus the program was designed using the Divide and Conquer technique to find the floor of x in a sorted array with \(O(\log n)\) time complexity.

OUTPUT:

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3.d) Two Elements sum to x

AIM:

To write a recursive program using the Divide and Conquer technique to find two elements in a sorted array whose sum equals the given value x.

ALGORITHM:

1. Start the program and read the sorted array, its size n, and the target value x.
2. Initialize two pointers, left at the beginning and right at the end of the array.
3. Calculate the sum of the elements at both pointers.
4. If the sum equals x, print the two elements; if smaller, move left forward; otherwise, move right backward.
5. Repeat recursively until a pair is found or the pointers meet; if no pair exists, print No.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int search(int arr[], int low, int high, long long target)
{
   if (low > high)
      return 0;
   int mid = low + (high - low) / 2;
   if (arr[mid] == target)
      return 1;
   if (arr[mid] < target)
      return search(arr, mid + 1, high, target);
   return search(arr, low, mid - 1, target);
}
int main()
{
   int n, x;
   scanf("%d", &n);
   int *arr = (int *)malloc((n > 0 ? n : 1) * sizeof(int));
   for (int i = 0; i < n; i++)
```

```
 scanf("%d", &arr[i]);
   scanf("%d", &x)
   for (int i = 0; i < n - 1; i++)
   {
      long long target = (long long)x - arr[i];
      if (search(arr, i + 1, n - 1, target))
      {
         printf("%d\n%d", arr[i], (int)target);
         free(arr);
         return 0;
      }
   }
   printf("No");
   free(arr);
   return 0;
}
```

RESULT:

Thus the program was designed to check whether two elements in a sorted array have a sum equal to x using a recursive approach, with \(O(n)\) time complexity.

OUTPUT:



	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

3.e) Implementation of Quick Sort

AIM:

To write a C program to implement the Quick Sort algorithm to sort a list of elements in ascending order.

ALGORITHM:

1. Start the program and read the number of elements n and the array elements.
2. Select a pivot element from the array.
3. Partition the array so that elements smaller than the pivot are placed before it and larger elements after it.
4. Recursively apply Quick Sort to the left and right partitions.
5. Print the sorted array and stop the program.

PROGRAM:

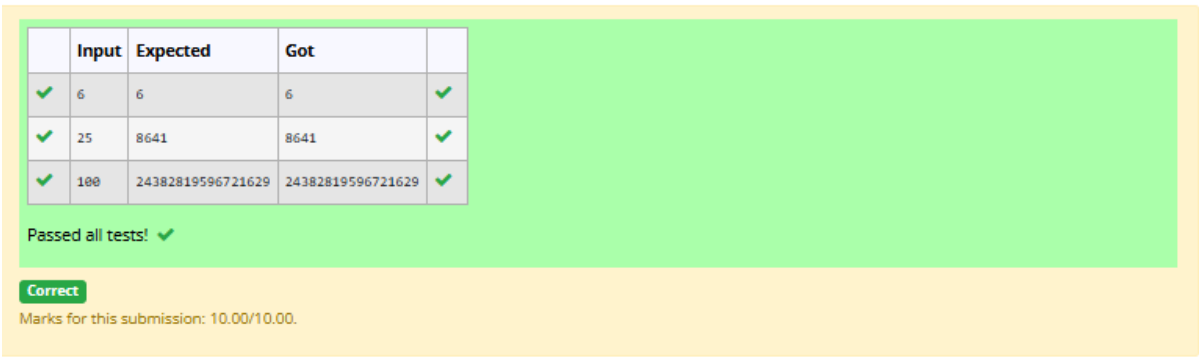
```
#include <stdio.h>
#include <stdlib.h>
void swap(int *a, int *b)
{
   int t = *a;
   *a = *b;
   *b = t;
}
int partition(int arr[], int low, int high)
{
   int pivot = arr[high];
   int i = low - 1;
   for (int j = low; j < high; j++)
   {
      if (arr[j] <= pivot)
      {
         i++;
         swap(&arr[i], &arr[j]);}}
```

```
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
int main()
{
    int n;
    scanf("%d", &n);
    int *arr = (int *)malloc((n > 0 ? n : 1) * sizeof(int));
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
    {
        if (i > 0)
            printf(" ");
        printf("%d", arr[i]);}
    free(arr);
    return 0;}
```

RESULT:

The Quick Sort algorithm was implemented successfully to sort the given list of elements in ascending order.

OUTPUT:



	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

DYNAMIC PROGRAMMING

4.a) DP-Playing with Numbers

AIM:

To write an efficient C program to find the number of possible ways to represent a positive integer n using the numbers 1 and 3.

ALGORITHM:

1. Start the program and read the positive integer n.
2. Initialize dp[0] = 1 and dp[1] = 1.
3. For each value i from 2 to n, calculate dp[i] = dp[i-1] + dp[i-3] if i >= 3; otherwise, set dp[i] = dp[i-1].
4. Repeat until dp[n] is calculated.
5. **Print dp[n] and stop the program.**

PROGRAM:

```
#include <stdio.h>

int main()
{
   int n;
   scanf("%d", &n);

   long long f[n + 4];
   f[0] = 1;
   f[1] = 1;
   f[2] = 1;

   for (int i = 3; i <= n; i++)
      f[i] = f[i - 1] + f[i - 3];

   printf("%lld", f[n]);

   return 0;
}
```

RESULT:

The algorithm was designed successfully to calculate the number of possible representations of n using 1 and 3 through dynamic programming

OUTPUT:

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

4.b) DP-Playing with chessboard

AIM:

To develop an efficient Dynamic Programming (DP) algorithm to find the maximum monetary value path from the top-left cell \((0,0)\) to the bottom-right cell \((n-1,n-1)\) of an \(n \times n\) chessboard, moving only right or down.

ALGORITHM:

1. Start the program and read the size n and the monetary values of the chessboard.
2. Initialize a DP table to store the maximum monetary sum for each cell.
3. Set dp[0][0] equal to the value of the starting cell.
4. For each cell, calculate the maximum sum from the top or left cell and add the current cell's value.
5. Print dp[n-1][n-1] as the maximum monetary path sum and stop.

PROGRAM:

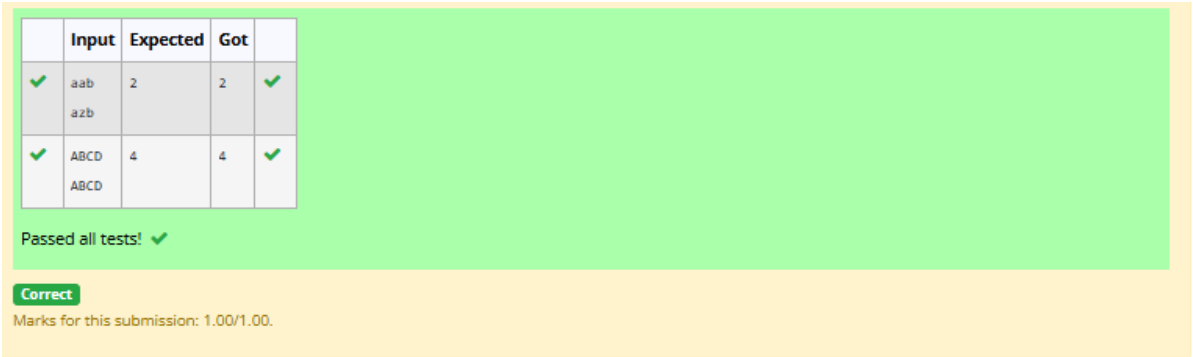
```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int n;
   scanf("%d", &n);
   long long **dp = (long long **)malloc(n * sizeof(long long *));
   for (int i = 0; i < n; i++)
   {
      dp[i] = (long long *)malloc(n * sizeof(long long));
      for (int j = 0; j < n; j++)
         scanf("%lld", &dp[i][j]);
   }
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         if (i == 0 && j == 0)
```

```
            continue;
        long long best;
        if (i == 0)
            best = dp[i][j - 1];
        else if (j == 0)
            best = dp[i - 1][j];
        else
            best = dp[i - 1][j] > dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1];
        dp[i][j] += best;
      }
    }
    printf("%lld", dp[n - 1][n - 1]);
    for (int i = 0; i < n; i++)
      free(dp[i]);
    free(dp);
    return 0;
}
```

RESULT:

The Dynamic Programming algorithm was designed successfully to find the maximum monetary value path from the top-left to the bottom-right cell, with a time complexity of \(O(n^2)\).

OUTPUT:



	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4.c) DP-Longest Common Subsequence

AIM:

To write a program using Dynamic Programming to find the length of the Longest Common Subsequence (LCS) between two strings, where the characters need not be contiguous.

ALGORITHM:

1. Start the program and read the two strings s1 and s2.
2. Create a DP table of size \((m+1)\times(n+1)\) and initialize the first row and column to zero.
3. If the characters match, set dp[i][j] = dp[i-1][j-1] + 1.
4. Otherwise, set dp[i][j] to the maximum of dp[i-1][j] and dp[i][j-1].
5. Print dp[m][n] as the length of the Longest Common Subsequence and stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
   char s1[5005], s2[5005];
   scanf("%5000s", s1);
   scanf("%5000s", s2);

   int n = strlen(s1);
   int m = strlen(s2);

   int **dp = (int **)malloc((n + 1) * sizeof(int *));
   for (int i = 0; i <= n; i++)
      dp[i] = (int *)calloc(m + 1, sizeof(int));

   for (int i = 1; i <= n; i++)
   {
      for (int j = 1; j <= m; j++)
```

```
 {
        if (s1[i - 1] == s2[j - 1])
            dp[i][j] = dp[i - 1][j - 1] + 1;
        else
            dp[i][j] = dp[i - 1][j] > dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1];
    }
  }

  printf("%d", dp[n][m]);

  for (int i = 0; i <= n; i++)
    free(dp[i]);
  free(dp);
  return 0;
}
```

RESULT:

The Dynamic Programming algorithm was designed successfully to find the length of the Longest Common Subsequence. For the given strings ggtabe and tgatasb, the LCS length is 4.

OUTPUT:

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

4.d) DP-Longest non-decreasing Subsequence

AIM:

To write a C program to find the length of the Longest Non-decreasing Subsequence (LNDS) in a given sequence using Dynamic Programming.

ALGORITHM:

1. Start the program and read the number of elements n and the sequence.
2. Initialize a DP array with 1 for every element.
3. Compare each element with the previous elements in the sequence.
4. If arr[i] >= arr[j], update dp[i] = max(dp[i], dp[j] + 1).
5. Print the maximum value in the DP array and stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n;
   scanf("%d", &n);

   int *arr = (int *)malloc((n > 0 ? n : 1) * sizeof(int));
   int *dp = (int *)malloc((n > 0 ? n : 1) * sizeof(int));

   for (int i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   int best = 0;
   for (int i = 0; i < n; i++)
   {
      dp[i] = 1;
      for (int j = 0; j < i; j++)
```

```
        {
  if (arr[j] <= arr[i] && dp[j] + 1 > dp[i])
                dp[i] = dp[j] + 1;
        }
        if (dp[i] > best)
            best = dp[i];
    }

    printf("%d", best);

    free(arr);
    free(dp);
    return 0;
}
```

RESULT:

The program was designed to find the length of the Longest Non-decreasing Subsequence using Dynamic Programming.

OUTPUT:

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5.a) Finding Duplicates-O(n^2) Time Complexity,O (1) Space Complexity

AIM:

To write a C program to find a duplicate element in a read-only array containing n integers between 1 and n.

ALGORITHM:

1. Start the program and read the number of elements n.
2. Read the n array elements.
3. Compare each element with the remaining elements in the array.
4. If two elements are equal, identify the repeated element.
5. Print the duplicate element and stop the program.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n;
   scanf("%d", &n);

   int *a = (int *)malloc(n * sizeof(int));
   for (int i = 0; i < n; i++) {
      scanf("%d", &a[i]);
   }

   int lo = 1, hi = n;
   while (lo < hi) {
      int mid = lo + (hi - lo) / 2;
      int count = 0;
      for (int i = 0; i < n; i++) {
         if (a[i] <= mid) count++;
      }
```

```
    if (count > mid)
            hi = mid;
        else
            lo = mid + 1;
    }

    printf("%d\n", lo);
    free(a);
    return 0;
}
```

RESULT:

The program was designed successfully to identify and print a repeated element in the given array.

OUTPUT:

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5.b) Finding Duplicates-O(n) Time Complexity,O (1) Space Complexity

AIM:

To write a C program to find one repeated number in a read-only array containing n integers between 1 and n

ALGORITHM:

1. Start the program and read the number of elements n.
2. Read the n elements into an array.
3. Compare each element with the remaining elements in the array.
4. If two elements are equal, identify the duplicate number.
5. Print the repeated element and stop the program.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n;
   scanf("%d", &n);

   int *seen = (int *)calloc(n + 1, sizeof(int));
   int ans = -1;

   for (int i = 0; i < n; i++) {
      int x;
      scanf("%d", &x);
      if (ans == -1 && x >= 1 && x <= n) {
         if (seen[x]) ans = x;
         else seen[x] = 1;
      }
   }
```

```
printf("%d\n", ans);
    free(seen);
    return 0;
}
```

RESULT:

The algorithm was designed successfully to find and print one duplicate element in the given array.

OUTPUT:

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5.c) Print Intersection of 2 sorted arrays-O(m* n)Time Complexity,O (1) Space Complexity

AIM:

To write a C program to find the intersection of two sorted arrays by identifying the elements common to both arrays.

ALGORITHM:

1. Start the program and read the number of test cases T.
2. For each test case, read the sizes and elements of both sorted arrays.
3. Initialize two pointers, i = 0 and j = 0.
4. If both elements are equal, print the element and increment both pointers; if the first is smaller, increment i, otherwise increment j.
5. Repeat until either array ends, then proceed to the next test case and stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int t;
   scanf("%d", &t);

   while (t--) {
      int n1, n2;

      scanf("%d", &n1);
      int *a = (int *)malloc(n1 * sizeof(int));
      for (int i = 0; i < n1; i++) scanf("%d", &a[i]);

      scanf("%d", &n2);
      int *b = (int *)malloc(n2 * sizeof(int));
      for (int i = 0; i < n2; i++) scanf("%d", &b[i]);

      int i = 0, j = 0, first = 1;
      while (i < n1 && j < n2) {
         if (a[i] < b[j]) {
```

```
 i++;
        } else if (a[i] > b[j]) {
            j++;
        } else {
            if (!first) printf(" ");
            printf("%d", a[i]);
            first = 0;
            int v = a[i];
            while (i < n1 && a[i] == v) i++;
            while (j < n2 && b[j] == v) j++;
        }
      }
      printf("\n");

      free(a);
      free(b);
   }
   return 0;
}
```

RESULT:

The program was designed successfully to find the common elements of two sorted arrays using the two-pointer technique.

OUTPUT:

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5.d) Print Intersection of 2 sorted arrays-O(m+ n)Time Complexity,O (1) Space Complexity

AIM:

To write a C program to find the intersection of two sorted arrays by identifying all elements that are common to both arrays.

ALGORITHM:

1. Start the program and read the number of test cases T.
2. For each test case, read the sizes and elements of both sorted arrays.
3. Initialize two pointers, i = 0 and j = 0.
4. If both elements are equal, print the element and increment both pointers. If the first element is smaller, increment i; otherwise, increment j.
5. Repeat until either array ends, process the remaining test cases, and stop.

PROGRAM:

```
#include <stdio.h>


int main() {
   int t;
   scanf("%d", &t);
   while (t--) {
      int n1, n2, i, j, first = 1;
      static int a[100000], b[100000];


      scanf("%d", &n1);
      for (i = 0; i < n1; i++) scanf("%d", &a[i]);
      scanf("%d", &n2);
      for (j = 0; j < n2; j++) scanf("%d", &b[j]);


      i = 0;
      j = 0;
      while (i < n1 && j < n2) {
         if (a[i] < b[j]) i++;
         else if (a[i] > b[j]) j++;
         else {
            if (!first) printf(" ");
```

```
            printf("%d", a[i]);
            first = 0;
            i++;
            j++;
          }
        }
        printf("\n");
    }
    return 0;
}
```

RESULT:

The program was designed successfully to find the common elements in two sorted arrays using the two-pointer technique.

OUTPUT:

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5.e) -Pair with Difference-O(n^ 2)Time Complexity,O (1) Space Complexity

AIM:

To write a C program to check whether two distinct indices in a sorted array exist such that the difference between their elements equals a given non-negative integer k

ALGORITHM:

1. Start the program and read the number of elements n and the sorted array.
2. Read the non-negative integer k.
3. Initialize two pointers, i = 0 and j = 1.
4. If A[j] - A[i] == k, return 1; if the difference is less than k, increment j; otherwise, increment i.
5. Repeat until a valid pair is found or the array ends. Print 1 if a pair exists; otherwise, print 0.

PROGRAM:

```
#include <stdio.h>

int main() {
   int n, k, i = 0, j = 1, found = 0;
   static int a[100000];

   scanf("%d", &n);
   for (int x = 0; x < n; x++) scanf("%d", &a[x]);
   scanf("%d", &k);

   while (i < n && j < n) {
      int d = a[j] - a[i];
      if (i != j && d == k) {
         found = 1;
         break;
      }
      if (d < k || i == j) j++;
      else i++;
   }

   printf("%d\n", found);
   return 0;}
```

RESULT:

The program was designed successfully to determine whether a pair of elements has the required difference using the two-pointer technique.

OUTPUT:

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5.f) Pair with Difference -O(n) Time Complexity,O (1) Space Complexity

AIM:

To write a C program to check whether two distinct elements in a sorted array have a difference equal to a given non-negative integer k.

ALGORITHM:

1. Start the program and read the number of elements n.
2. Read the sorted array elements and the value of k.
3. Initialize two pointers, i = 0 and j = 1.
4. Compare A[j] - A[i] with k; if equal, print 1. If smaller, increment j; otherwise, increment i.
5. Continue until a pair is found or the array ends. Print 0 if no pair exists.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n, k, i = 0, j = 1, found = 0;
   scanf("%d", &n);
   int *a = (int *)malloc(n * sizeof(int));
   for (int x = 0; x < n; x++) scanf("%d", &a[x]);
   scanf("%d", &k);
   while (i < n && j < n) {
      long long d = (long long)a[j] - a[i];
      if (i != j && d == k) {
         found = 1;
         break;
      }
      if (d < k || i == j) j++;
      else i++;
   }
   printf("%d\n", found);
   free(a);
   return 0;
}
```

RESULT:

The program was designed successfully to check whether a pair with the required difference exists using the two-pointer technique, with \(O(n)\) time complexity.